

El patrón Proxy se ve en tu proyecto principalmente en estas 4 partes:

1. La interfaz CuentaBancaria

Esta interfaz define las operaciones que se pueden hacer:

```
public interface CuentaBancaria {  
    String consultarSaldo(String pin);  
    String depositar(String pin, double monto);  
    String retirar(String pin, double monto);  
    String transferir(String pin, String cuentaDestino, double monto);  
}
```

Esta interfaz representa el **Subject** del patrón Proxy.

Sirve para decir:

Estas son las operaciones bancarias disponibles.

2. La clase CuentaBancariaReal

Aquí está el objeto real.

```
public class CuentaBancariaReal {
```

Esta clase hace las operaciones verdaderas:

```
    public String consultarSaldo()  
    public String depositar(double monto)  
    public String retirar(double monto)  
    public String transferir(CuentaBancariaReal destino, double monto)
```

Esta clase representa el **RealSubject**.

Su función es manejar:

- saldo,
- depósitos,
- retiros,
- transferencias.

Pero no valida PIN ni seguridad.

3. La clase CuentaBancariaProxy

Aquí es donde realmente se ve el patrón Proxy.

```
public class CuentaBancariaProxy implements CuentaBancaria {
```

Esta línea es muy importante porque significa:

El Proxy implementa la misma interfaz que el servicio bancario.

Luego dentro tiene una referencia al objeto real:

```
private CuentaBancariaReal cuentaReal;
```

Esto significa:

El Proxy conoce a la cuenta real y puede delegarle operaciones.

También tiene:

```
private CuentaBancariaReal cuentaDestino;
```

para manejar transferencias.

4. El Proxy valida antes de llamar al objeto real

Por ejemplo, en consultar saldo:

```
@Override
```

```
public String consultarSaldo(String pin) {  
    if (!autenticar(pin)) {  
        return "Acceso denegado.";  
    }  
}
```

```
    mensajesProxy += "[Proxy] Delegando consulta a CuentaBancariaReal...\n";  
    return cuentaReal.consultarSaldo();  
}
```

Aquí se ve clarísimo el patrón Proxy:

Primero:

```
autenticar(pin)
```

Después recién:

```
cuentaReal.consultarSaldo()
```

Eso significa:

Usuario → Proxy → Cuenta Real

5. Ejemplo con retiro

```

@Override
public String retirar(String pin, double monto) {
    if (!autenticar(pin)) {
        return "Retiro denegado.";
    }

    if (!validarMonto(monto)) {
        return "Retiro rechazado.";
    }

    if (!validarSaldo(monto)) {
        return "Retiro rechazado por saldo insuficiente.";
    }

    mensajesProxy += "[Proxy] Delegando retiro a CuentaBancariaReal...\n";
    return cuentaReal.retirar(monto);
}

```

Aquí el Proxy hace varias validaciones:

```

autenticar(pin)
validarMonto(monto)
validarSaldo(monto)

```

Y solo si todo está correcto, llama a:

```
cuentaReal.retirar(monto)
```

Esa es la esencia del patrón Proxy.

6. Ejemplo con transferencia

```

@Override
public String transferir(String pin, String numeroCuentaDestino, double monto) {
    if (!autenticar(pin)) {
        return "Transferencia denegada.";
    }

    if (!validarCuentaDestino(numeroCuentaDestino)) {
        return "Transferencia rechazada. Cuenta destino inválida.";
    }

    if (!validarMonto(monto)) {

```

```

        return "Transferencia rechazada. Monto inválido.";
    }

    if (!validarSaldo(monto)) {
        return "Transferencia rechazada. Saldo insuficiente.";
    }

    if (!validarLimiteDiario(monto)) {
        return "Transferencia rechazada. Supera el límite diario.";
    }

    mensajesProxy += "[Proxy] Delegando transferencia a CuentaBancariaReal...\n";

    totalTransferidoHoy += monto;
    return cuentaReal.transferir(cuentaDestino, monto);
}

```

Aquí se ve todavía mejor.

El Proxy revisa:

- PIN,
- cuenta destino,
- monto,
- saldo,
- límite diario.

Y recién después llama a:

```
cuentaReal.transferir(cuentaDestino, monto)
```

7. La interfaz gráfica FormBanco

En FormBanco, el usuario no usa directamente CuentaBancariaReal.

Usa:

```
private CuentaBancariaProxy proxyBanco;
```

Y cuando presiona botones, llama al Proxy:

```
String resultado = proxyBanco.consultarSaldo(pin);
```

o:

```
String resultado = proxyBanco.depositar(pin, monto);
```

o:

```
String resultado = proxyBanco.retirar(pin, monto);
```

o:

```
String resultado = proxyBanco.transferir(pin, cuentaDestino, monto);
```

Eso demuestra que la interfaz trabaja con el Proxy, no con la cuenta real.

Flujo completo

FormBanco

↓

CuentaBancariaProxy

↓

Validaciones de seguridad

↓

CuentaBancariaReal

La parte más importante para mostrar al docente

Esta parte:

```
if (!autenticar(pin)) {  
    return "Acceso denegado."  
}
```

```
return cuentaReal.consultarSaldo();
```

Porque ahí se ve que:

1. El Proxy recibe la solicitud.
2. El Proxy valida.
3. Si está correcto, delega al objeto real.
4. Si está mal, bloquea el acceso.